Cache Behavior Prediction Using ML

Machine Learning (SE-206)

Group Details

Name: Aaradhya Bhardwaj
Roll: 24/SE/005

Name: Aditya Minz
Roll: 24/SE/013

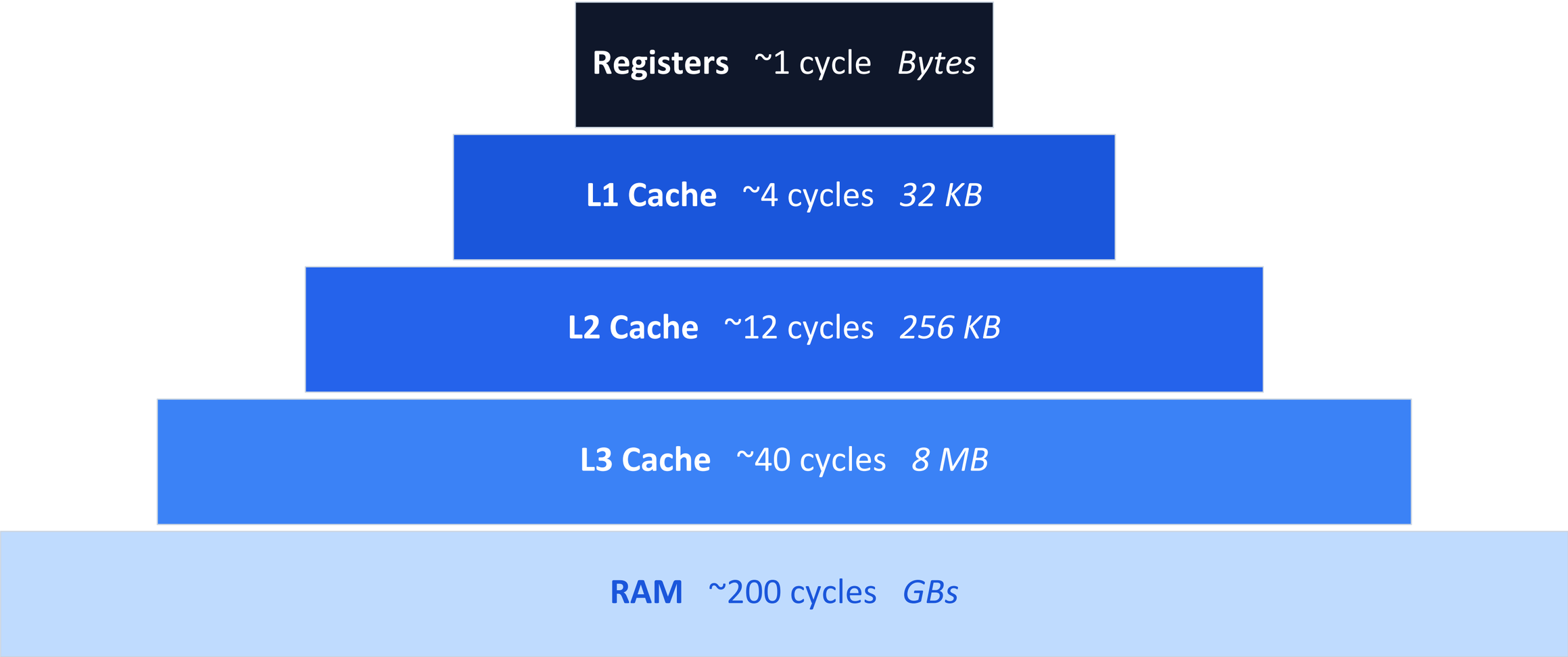
Outline

- Motivation
- Introduction
- Dataset
- Methodology
- Algorithms
- Techniques
- Performance
- Model Comparison
- Confusion Matrices
- Feature Importance
- Limitations
- Conclusion
- Future Work
- References

Motivation

- Modern CPUs are orders of magnitude faster than RAM — cache efficiency is the bottleneck in program performance
- Identifying cache-hostile code today requires running the program and measuring with profiling tools — slow and not scalable
- No existing tool predicts cache behavior from source code alone, without execution
- Machine Learning can learn the relationship between code structure and cache behavior statically
- Goal: build a model that flags cache-hostile programs before they are run

Why Cache Matters



A cache miss costs 50× more than a cache hit

Cache-Friendly vs Cache-Hostile Access

Sequential Access

→ → → → → → →

- Reads consecutive memory
- Prefetcher predicts next address
- **Cache miss rate: ~0.5%**

Random Access

↗ ↙ ↘ ↗ ↙ ↗

- Jumps unpredictably in memory
- Prefetcher cannot predict
- **Cache miss rate: ~25–35%**

Problem Statement

Given a C program's static structural parameters:

- Array size, stride, access pattern type
- Working set size relative to cache levels
- Spatial and temporal locality characteristics

Predict:

cache_friendly = 1 or **cache_hostile = 0**

WITHOUT executing the program

Dataset: 30,000 synthetic C programs | 10 memory access patterns | ~33 static features per program

Dataset

Source: Synthetically generated — 30,000 C programs

Memory Access Patterns (10 types)

Sequential, Strided, Random, Matrix Row-Major,
Matrix Column-Major, Matrix Tiled, Pointer Chasing,
Histogram, Gather-Scatter, Working Set

Key Features (33 static features)

- array_size, stride, matrix_size, iters
- fits_in_L1/L2/L3, L1/L2/L3_ratio
- spatial_locality_score, temporal_locality_score
- access_regularity, stride_in_cache_lines

Labeling: Per-pattern median miss rate split → balanced 50% friendly / 50% hostile

Pipeline

Generate C Programs
generate_programs.py — 30,000 programs, 10 patterns

Compile
compile_programs.sh

Measure Cache Behavior
run_perf.py

Build Dataset
generate_dataset.py

Train ML Models
train.py

Evaluate & Compare
Accuracy, Precision, Recall, F1 — confusion matrices

Algorithm 1: Logistic Regression

- Linear classifier: finds a hyperplane separating the two classes
- Uses sigmoid function to output probability between 0 and 1
- Predicts cache_friendly=1 if $P(y=1|X) \geq 0.5$
- Features scaled with StandardScaler before training
- max_iter = 1000

Results

Accuracy
0.7701

F1 Score
0.7742

Conclusion:

- *Weakest model; cache behavior boundaries are non-linear; logistic regression cannot capture feature interactions*

Algorithm 2: K-Nearest Neighbours

- Instance-based learner — no explicit training phase
- Classifies a program by looking at its 7 nearest neighbours in feature space and taking majority vote
- Distance measured in Euclidean space (features scaled)
- k = 7

Results

Accuracy
0.9016

F1 Score
0.9012

Conclusion:

- *Strong performance; similar programs (same pattern, similar array size) cluster naturally in feature space*

Algorithm 3: Support Vector Machine (SVM)

- Finds the maximal margin hyperplane between classes
- Uses RBF (Radial Basis Function) kernel to handle non-linear decision boundaries
- Maps features into higher-dimensional space where classes become linearly separable
- Features scaled with StandardScaler

Results

Accuracy
0.8578

F1 Score
0.8469

Conclusion:

- *Moderate performance; RBF kernel handles non-linearity but struggles with the mix of binary flags and continuous features at 30k samples*

Algorithm 4: Decision Tree

- Recursively splits data on the feature that maximizes information gain at each node
- Learns rules like: "if stride_in_cache_lines > 1 AND L2_ratio > 1 → hostile"
- max_depth = 12, min_samples_leaf = 5

Results

Accuracy
0.9115

F1 Score
0.9093

Conclusion:

- *Best single-model result; cache behavior is inherently rule-based, which decision trees capture perfectly*

Algorithm 5: Random Forest

- Ensemble of 200 Decision Trees
- Each tree trained on a random subset of data (bagging) and a random subset of features at each split
- Final prediction = majority vote of all 200 trees
- Reduces variance compared to a single tree
- n_estimators = 200, min_samples_leaf = 2

Results

Accuracy
0.9115

F1 Score
0.9093

Conclusion:

- *Tied with Decision Tree; clean synthetic data means no noise to reduce; ensemble gain is minimal here*

Techniques Used

Data Preprocessing

- Removed leakage features (cache_misses, miss_rate, wall_time)
- Dropped null/constant columns
- StandardScaler applied for distance-based models (KNN, SVM, LR)

Feature Engineering

- fits_in_L1/L2/L3 — binary flags vs 32KB/256KB/8MB thresholds
- spatial_locality_score — how sequentially memory is accessed
- stride_in_cache_lines — whether stride crosses cache line boundary

Model Training & Testing

- 5-Fold Stratified Cross-Validation
- Stratified = each fold preserves the 50/50 class ratio

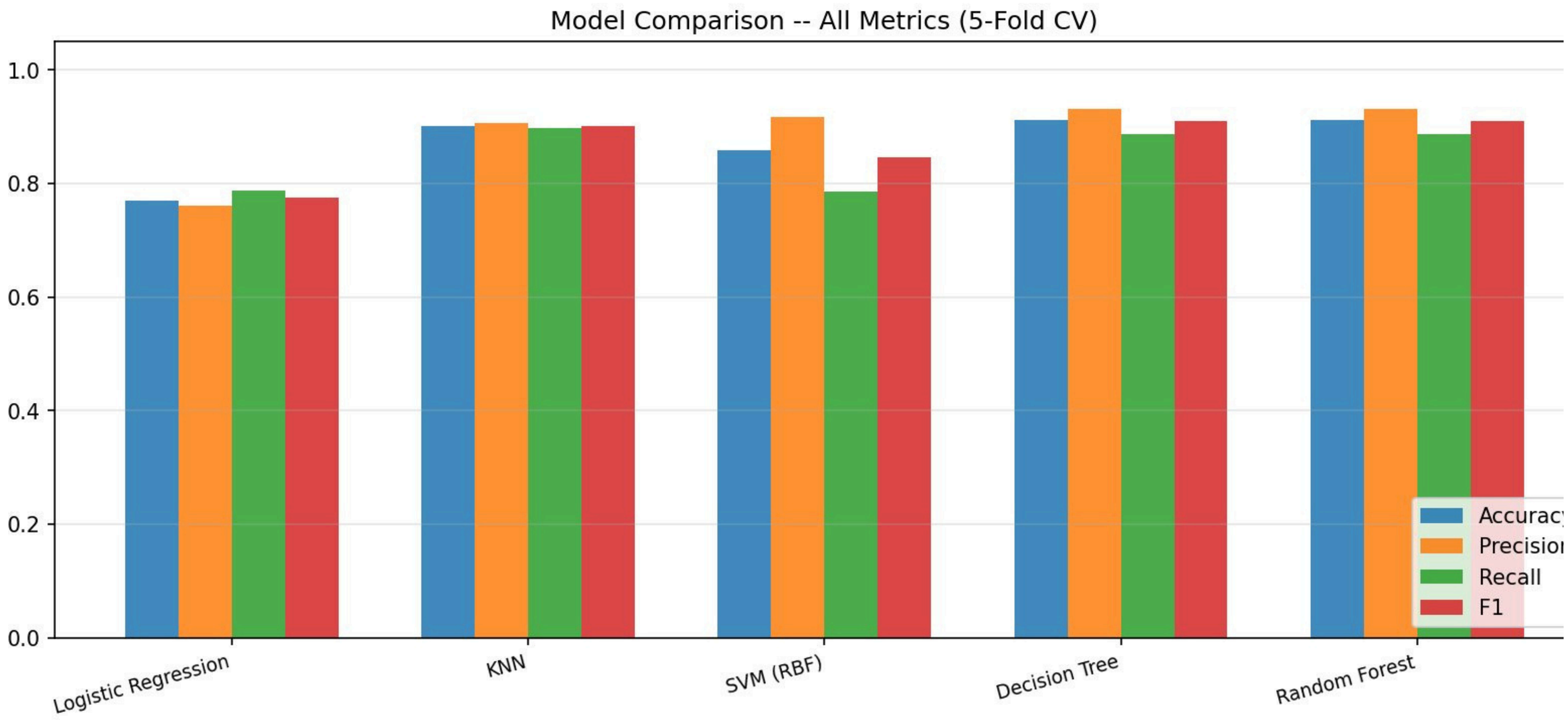
Evaluation Metrics

- Accuracy, Precision, Recall, F1-Score
- Confusion Matrix for each model

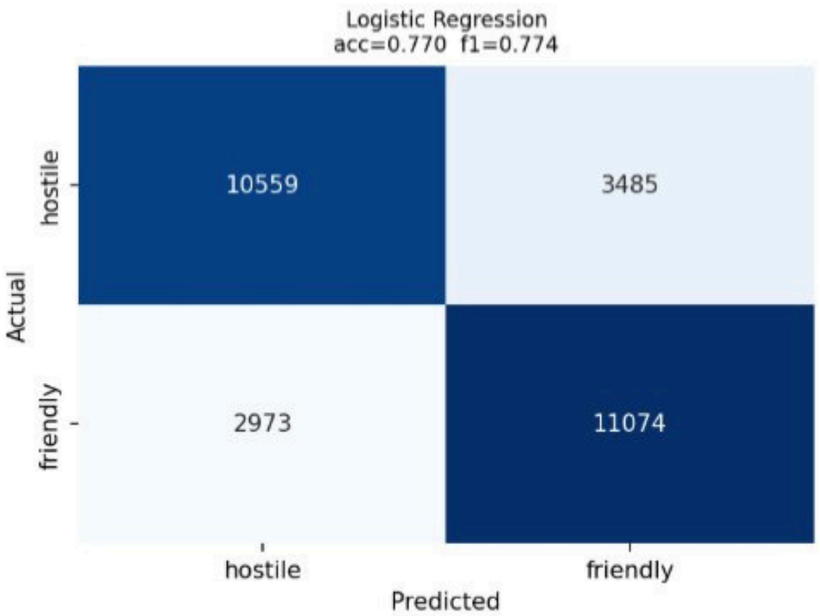
Performance

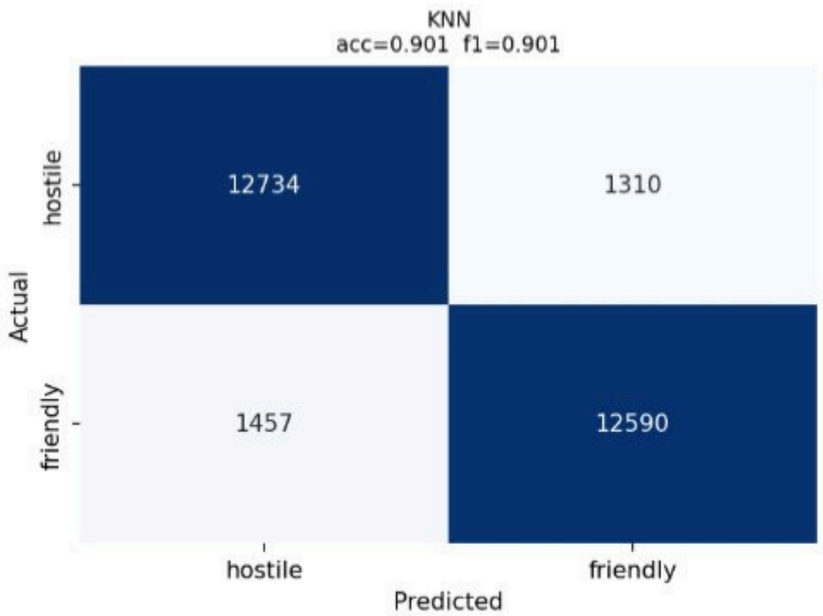
Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.7701	0.7756	0.7742	0.7742
KNN (k=7)	0.9016	0.9018	0.9012	0.9012
SVM (RBF kernel)	0.8578	0.8608	0.8469	0.8469
Decision Tree	0.9115	0.9098	0.9093	0.9093
Random Forest	0.9115	0.9098	0.9093	0.9093

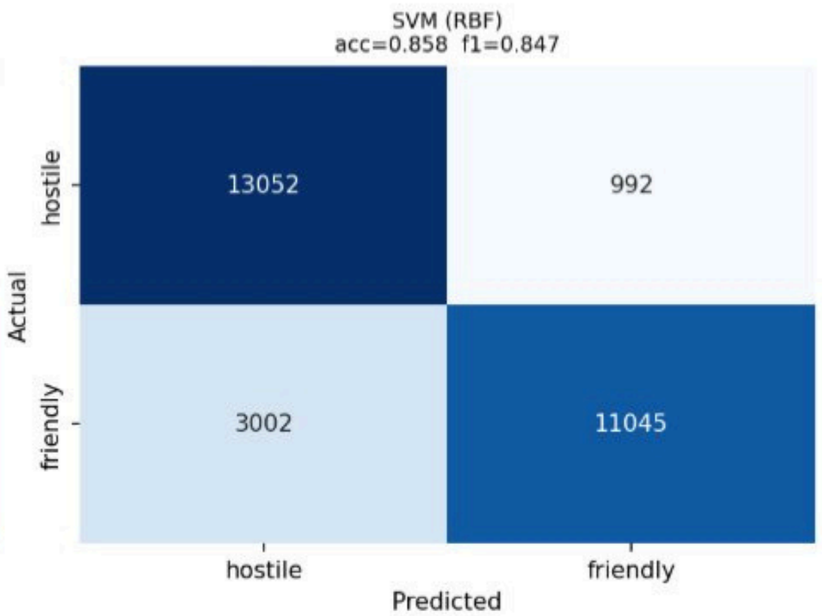
Model Comparison

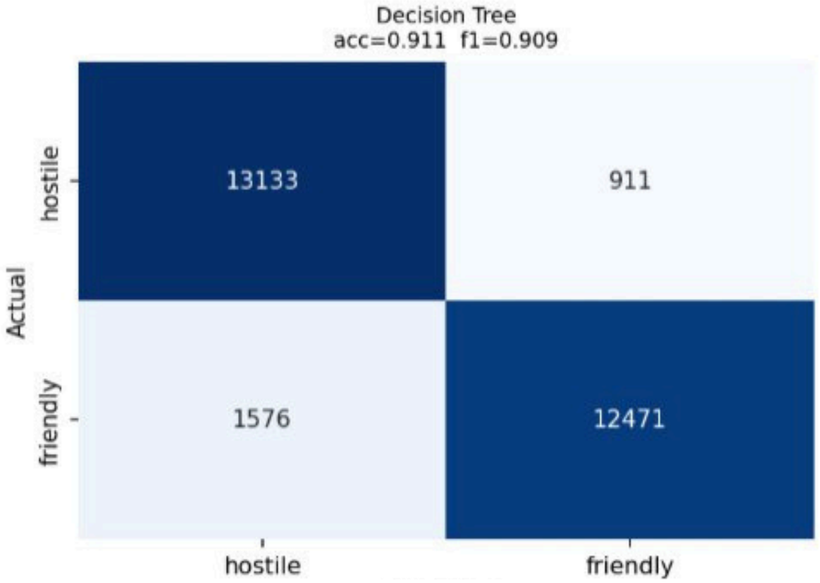


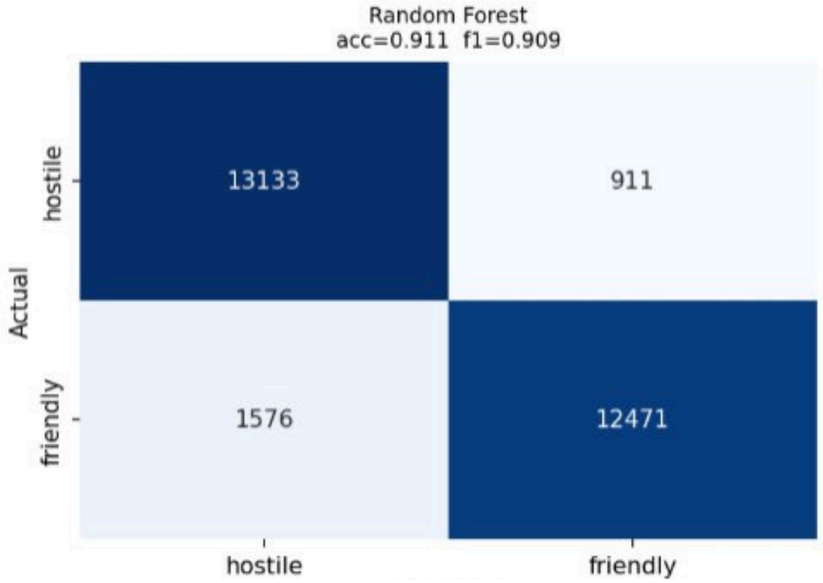
Confusion Matrices



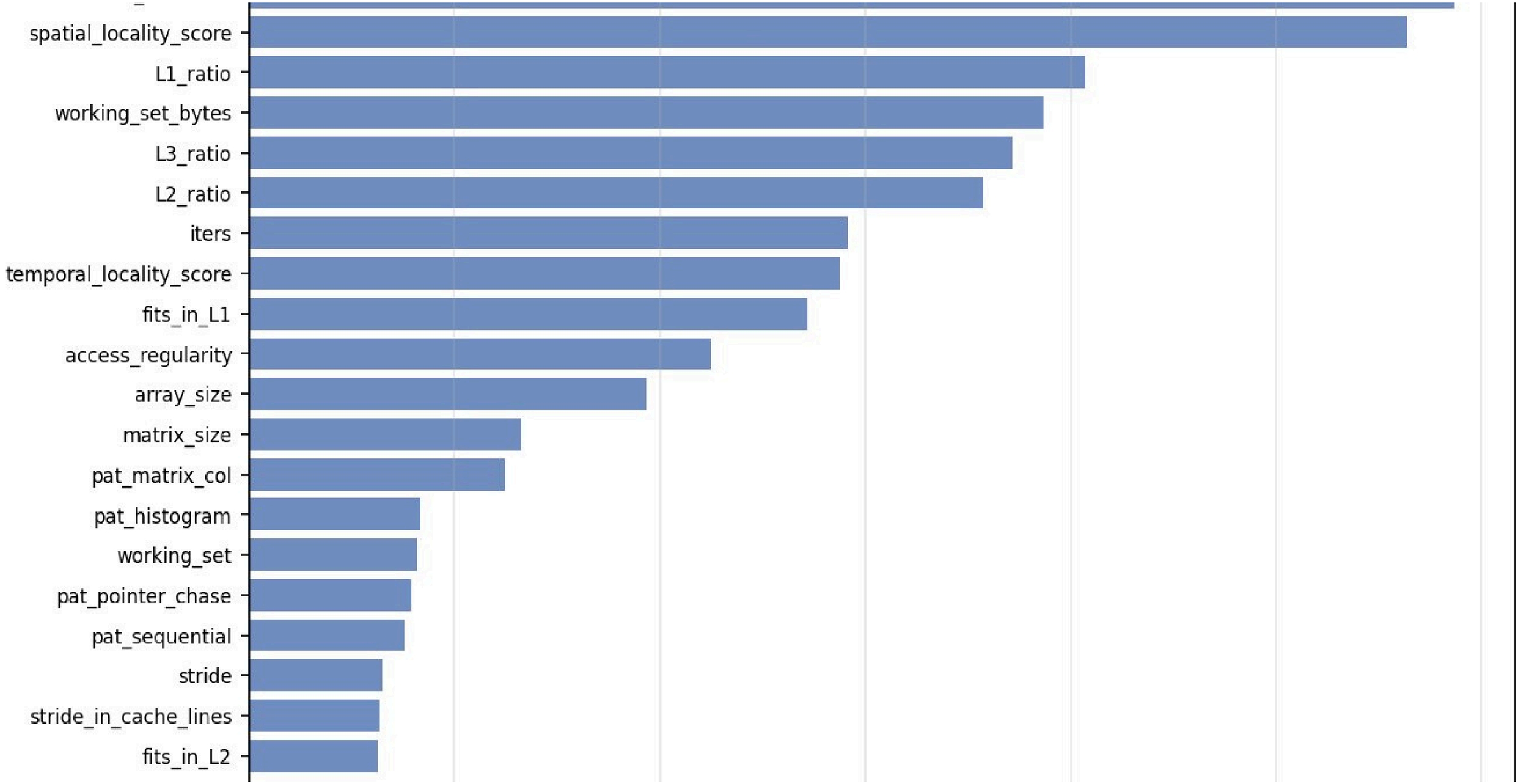








Feature Importance (Random Forest; Top 20)



Limitations

- **Synthetic data** only: programs are generated with clean, isolated patterns. Real-world code has mixed patterns which the model has not seen.
- **Static features are incomplete**: cache behavior also depends on runtime factors (data values, OS scheduling, hardware prefetcher state) that cannot be captured statically.
- **Binary classification** only: the model outputs friendly/hostile but does not quantify how bad the miss rate actually is.
- **Decision Tree = Random Forest** : on clean synthetic data, ensembling provides no benefit. On real-world noisy data this gap would reappear.

Conclusion

- Built a complete end-to-end pipeline: C program generation → compilation → perf measurement → dataset construction → ML training
- Demonstrated that cache behavior can be predicted from static code features alone with up to 91% accuracy
- Tree-based models (Decision Tree, Random Forest) outperformed linear models (Logistic Regression, SVM), confirming that cache behavior boundaries are non-linear
- Key insight: engineered features (spatial_locality_score, fits_in_L1, stride_in_cache_lines) carry far more predictive signal than raw parameters alone
- The 77% → 91% accuracy gap between Logistic Regression and Random Forest illustrates the value of non-linear classifiers for systems-level prediction tasks

Future Work

- Test on real-world C/C++ programs (open source projects, benchmarks like SPEC CPU) to validate on non-synthetic data
- Extend to multi-class prediction — not just friendly/hostile but L1-friendly, L2-friendly, L3-friendly, hostile
- Add compiler optimization levels (O0, O1, O2, O3) as a variable — optimization changes access patterns significantly
- Use Abstract Syntax Tree (AST) features extracted directly from source code instead of hand-crafted parameters
- Cross-architecture testing — ARM vs x86 have different cache geometries; test if the model generalizes

References

[1] T. M. Mitchell, Machine Learning, 1st ed. New York, NY, USA: McGraw-Hill, 1997.
[2] E. Alpaydin, Introduction to Machine Learning, 3rd ed. Cambridge, MA, USA: MIT Press, 2014.
[3] U. Drepper, "What Every Programmer Should Know About Memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
[5] "perf: Linux profiling with performance counters," The Linux Kernel Archives. [Online]. Available: https://perf.wiki.kernel.org